**Name: Shalini Punithan  Roll No: 241801258**

# PODS

# EXPERIMENT 3 – MODEL PLANNING AND BUILDING

## Learning Objectives

The objective of this experiment is to learn the end-to-end machine learning process using the Advertising dataset. It includes data preprocessing, visualization, training Linear Regression, Polynomial Regression, and SVR models, evaluating their performance, and applying KMeans clustering for data grouping.

## Libraries Used

The libraries used are **Pandas** for data handling, **NumPy** for numerical operations, **Matplotlib** and **Seaborn** for visualization, and **Scikit-learn (sklearn)** for data splitting, regression models, clustering, and performance evaluation.

**CODE:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error

uploaded = files.upload()

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

if 'Unnamed: 0' in df.columns:
    df.drop('Unnamed: 0', axis=1, inplace=True)
```

```
filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

if 'Unnamed: 0' in df.columns:
    df.drop('Unnamed: 0', axis=1, inplace=True)

print(df.head())
print(df.info())
print(df.isnull().sum())
print(df.dtypes)
print(df.describe())

X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']

sns.pairplot(df)
plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])

poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)

svr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf'))
```

```
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])

poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)

svr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf'))
])

svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    print(f"\n{name}")
    print("MAE :", round(mae,3))
    print("MSE :", round(mse,3))
    print("RMSE:", round(rmse,3))
    return mae, mse, rmse

mae_lr, mse_lr, rmse_lr = evaluate("Linear Regression", y_test, y_pred_lr)
mae_poly, mse_poly, rmse_poly = evaluate("Polynomial Regression", y_test, y_pred_poly)
mae_svr, mse_svr, rmse_svr = evaluate("Support Vector Regression", y_test, y_pred_svr)

models = ["Linear Regression", "Polynomial Regression", "SVR"]
rmse = [rmse_lr, rmse_poly, rmse_svr]

plt.figure(figsize=(8,5))
plt.bar(models, rmse, color=["blue","green","red"])
plt.title("RMSE Comparison")
plt.xlabel("Models")
plt.ylabel("RMSE")

for i, v in enumerate(rmse):
    plt.text(i, v+0.05, f"{v:.2f}", ha="center")

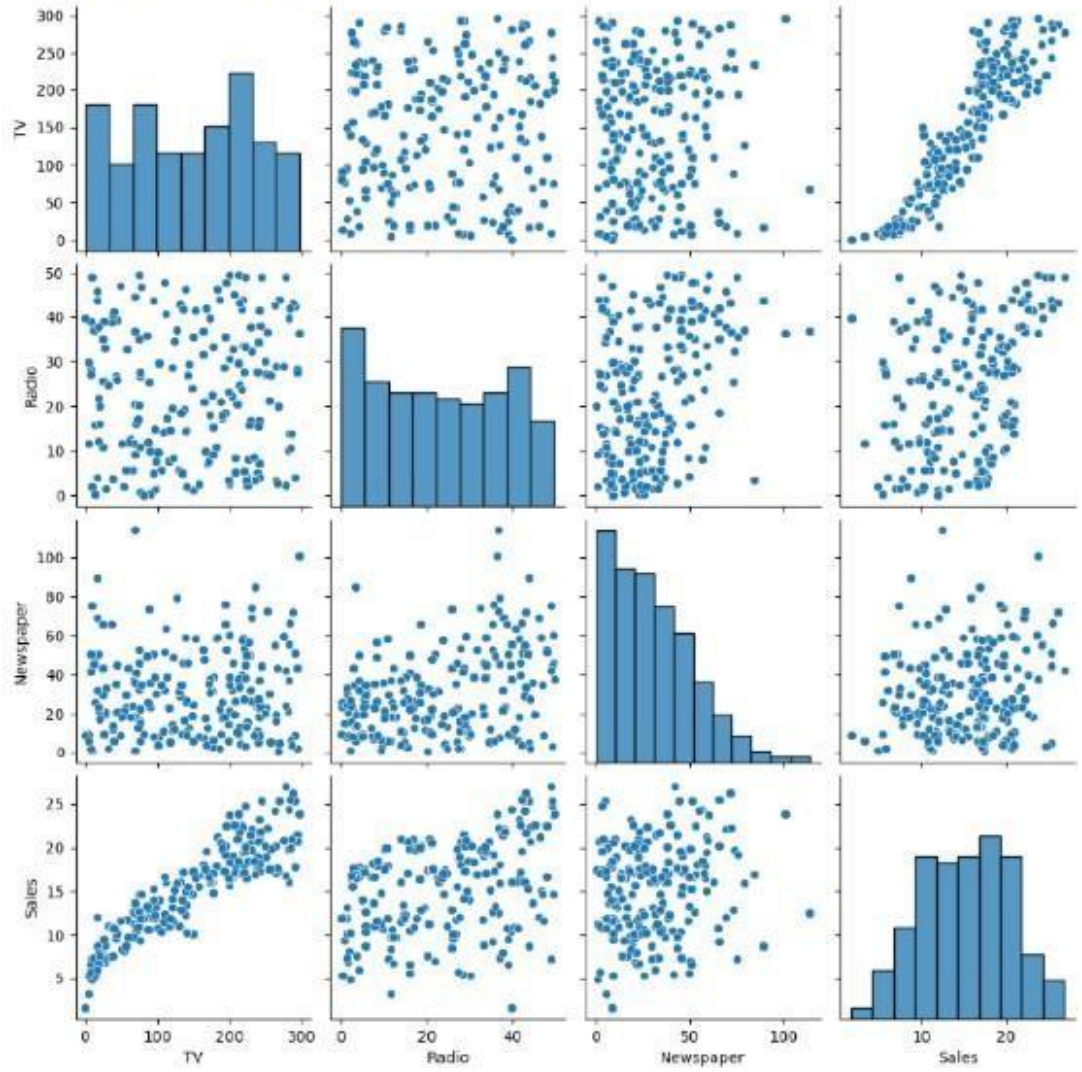
plt.show()

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X)

plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x="TV", y="sales", hue="cluster", palette="Set1", s=100)
plt.title("K-Means Clustering")
plt.show()
```

```
Choose files  advertising.csv
advertising.csv(text/csv) - 4082 bytes, last modified: 28/07/2026 - 100% done
Saving advertising.csv to advertising.csv
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
None
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

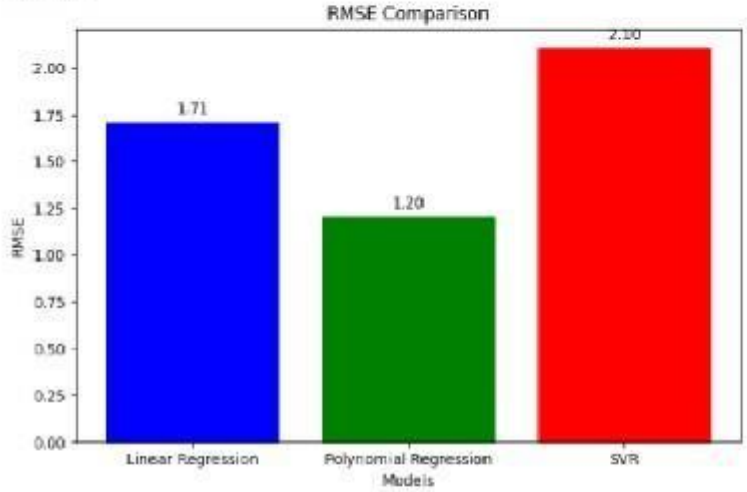
max 290.400000 49.000000 114.000000 27.000000

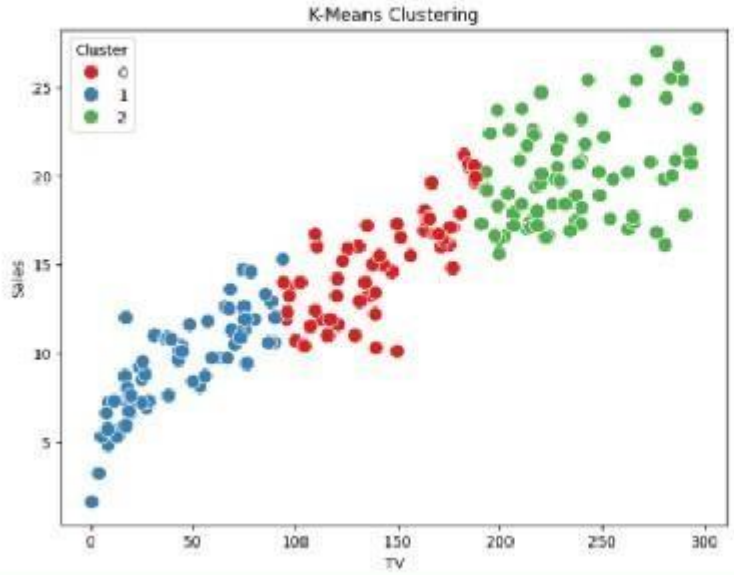


```
Linear Regression
MAE : 1.275
MSE : 2.908
RMSE: 1.705

Polynomial Regression
MAE : 0.983
MSE : 1.443
RMSE: 1.201

Support Vector Regression
MAE : 1.555
MSE : 4.421
RMSE: 2.103
```




## Learning Outcomes

After completing this experiment, students will be able to preprocess data, build and compare different regression models, evaluate their accuracy using error metrics, perform K-Means clustering, and visualize results using Python libraries.